



Functional/Logic Programming in Verse

Lennart Augustsson, Koen Claessen,
Simon Peyton Jones, Tim Sweeney, Jan Vitek, **Stephanie Weirich**

Epic Games

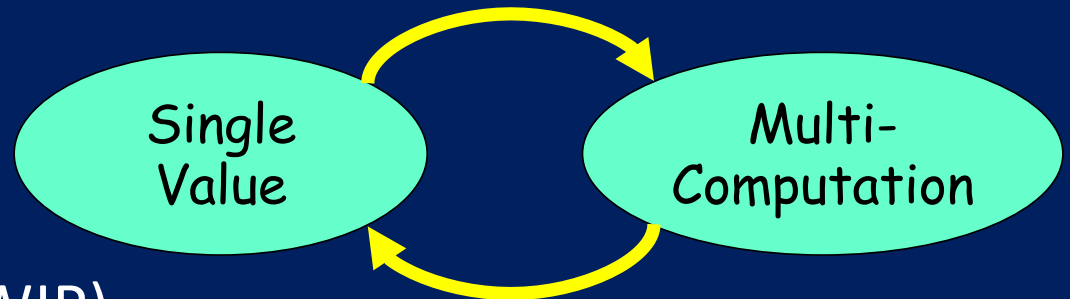
August 2026

The Verse Language

- Verse is a **functional logic language**.
- Verse is **lenient** but not strict:
 - Like strict: everything gets evaluated in the end
 - Like lazy: functions can be called before the argument has a value
- Verse is a **declarative language**: a variable names a single value, not a cell whose value changes over time.
- Verse unifies **expressions** and **patterns**.
- Verse has an unusual type system that unifies **types** and **expressions**.
- Verse has an **effect system**, rather than using monads.

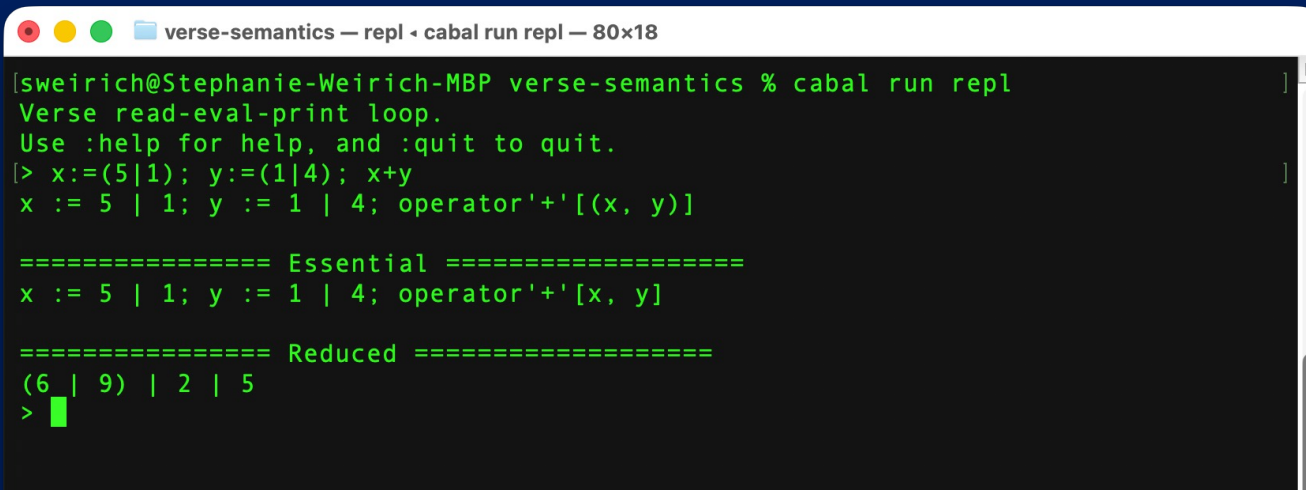
Today

- Some Verse features, by example
 - Computation with multiple results: **Choice** and Failure
 - Values that reify choice: **Arrays**
 - **Functions** with explicit domains
 - **Types** as identity functions
- Denotational Semantics (WIP)
 - How to make sense of all this?



Learn more about Verse

- This talk was derived from Simon Peyton Jones' lectures at OPLSS 2026. Videos, slides, and notes are available.
- Prototype available
<https://github.com/augustss/verse-semantics>



```
verse-semantics — repl • cabal run repl — 80x18
[sweirich@Stephanie-Weirich-MBP verse-semantics % cabal run repl
Verse read-eval-print loop.
Use :help for help, and :quit to quit.
[> x:=(5|1); y:=(1|4); x+y
x := 5 | 1; y := 1 | 4; operator'+'[(x, y)]

===== Essential =====
x := 5 | 1; y := 1 | 4; operator'+'[x, y]

===== Reduced =====
(6 | 9) | 2 | 5
> 
```

At first sight, pretty conventional

Typed function
declaration

```
f(x:int) := x+1;
```

Despite the "!=" notation,
f is not mutable!
x:=e1; e2
is like
let x=e1 in e2

```
f[7] + f[8]
```

";" for sequencing

Function call
(square brackets)

At first sight, pretty conventional

- Of course, Verse has a lambda form too:

```
f1 (x:int) := x+1;  
f2 := function (x:int) <ucceeds> {x+1}  
f3 := (x:int) => x+1
```

- All three do the same thing

Effect annotation,
this function must
be total

Nontraditional Binding

```
x+1    # An occurrence of x
3=x    # An occurrence of x
x=3    # An occurrence of x
x:=3   # A binding of x, equal to 3
x:int  # A binding of x, must be some int
```

- $(x:e)$ and $(x:=e)$ bring x into scope throughout the **current block**, and can appear **anywhere**, not just at the beginning
- plain x is an occurrence of x

```
y:=x+1; x:int; (z | 2*y)
```

means

```
 $\exists x \ y. \ y=x+1; \ x:int; \ (z \mid 2*y)$ 
```

**Choice and Failure are
integral to computation**

Choice and failure

- An expression can
 - **fail**, without a value
 - **succeed**, returning a single value
 - **iterate**, returning several values

Choice is NOT commutative
5 **and then** 6

<code>fail</code>	<code># Fails</code>
<code>5</code>	<code># Succeeds, returning 5</code>
<code>5 6</code>	<code># Iterates, returning 5 then 6</code>
<code>5 fail</code>	<code># Succeeds, returning 5</code>
<code>fail 5</code>	<code># Succeeds, returning 5</code>
<code>4..8</code>	<code># Returns 4 then 5 ... 8</code>
<code>4 5 6 7 8</code>	<code># Same</code>

Conditionals: no booleans

An old idea: [SNOBOL, Icon, 1979]

```
if (e) { e1 } else { e2 }
```

- Returns e1 if e **succeeds**
 - “Succeeds” = returns one or more values
- Returns e2 if e **fails**
 - “Fails” = returns zero values

“Fails” is not a Bad Thing; it's like False in Haskell or ML

```
if (x>20) { e1 } else { e2 }
```

- (x>20) **succeeds** if x>20, **returning x**,
fails if x≤20
- No Boolean type

Most predicates do this: they are partial identity functions

Choice and iteration

Sequencing takes
Cartesian product.
Like list comprehensions in
Haskell

```
x := (5 | 1) ; y := (1 | 4) ; x+y  
# Returns 6 then 9 then 2 then 5
```

Filtering
 $x > 2$ fails when $x \leq 2$

```
x := (5 | 1) ; y := (1 | 4) ; x > 2 ; x+y  
# Returns 6 then 9
```

```
x := (5 | 1) ; y := (1 | 4) ; x = y ; x+y  
# Returns 2
```

Verse uses "=" for equality
Fails when not equal.
Often called "unification"!

Values that reify choice:
Arrays (i.e. tuples)

Arrays (aka tuples)

Arrays and tuples are names for the same thing in Verse

```
(4,2,7)           # An array of 3 elements

a := (4,2,7);      # Bind (4,2,7) to a
a[0]              # Index into a, returning 4
```

Arrays and choice

```
(4, (1|2), 7)
(4, fail, 7)
((5|4), (1|2), 7)
```

Arrays and choice

```
(4,2,7)           # An array of 3 elements

a := (4,2,7);     # Bind (4,2,7) to a
a[0]              # Index into a, returning 4
```

Arrays and choice

```
(4, (1|2), 7)      # (4,1,7) | (4,2,7)
(4, fail, 7)       # fail
((5|4), (1|2), 7)  # (5,1,7) | (5,2,7) |
                  # (4,1,7) | (4,2,7)
```

Just cartesian
product, like list
comprehensions in
Haskell

Creating arrays: `for` turns choice into data

```
for{4|2|7}           # The array (4,2,7)
for{1..10}           # The array (1,2,3,..., 10)
for{x:=1..10; x*x}   # The array (1,4,9,..., 100)
for{x:=1..10; even[x]} # The array (2,4,..., 10)
for{fail}            # The array ()
```

- 'for' takes a choice of values and makes an array of them
- Always returns **exactly one** array, even if there are no values
- **Requires ordered choice**

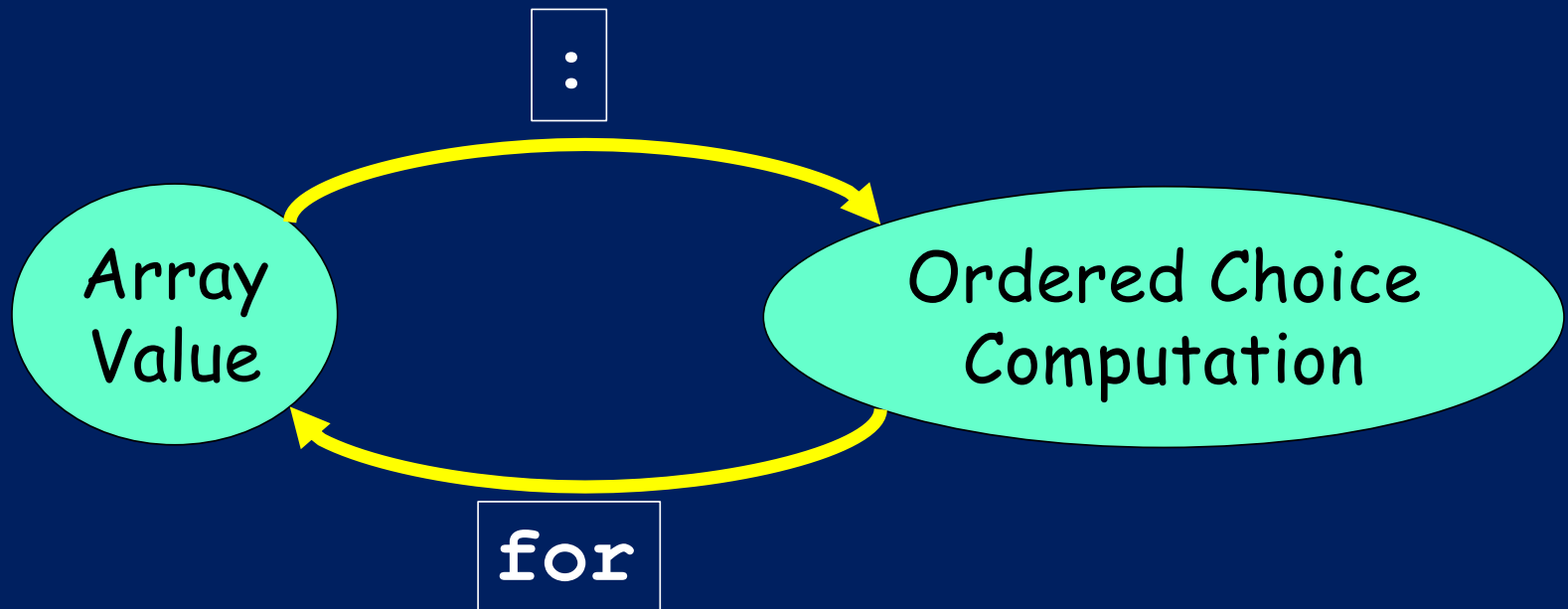
From choice to arrays and back: magic colon

for{4|2|7}

Returns the array (4,2,7)

: (4,2,7)

Iterates like 4|2|7



Creating arrays: `for` turns choice into data

- Making new arrays from old ones

```
for {i := 0 .. Length[a] - 1; a[i] + 7}
    # Add 7 to each
    # element of a
```

- Here is a nicer way to say the same thing

```
for {x: a; x + 7}
```

Functions with explicit domains

Arrays are closed, finite functions

```
(4,2,7)                # An array of 3 elements  
  
a := (4,2,7)           # Bind (4,2,7) to a  
a[0]                   # Index into a  
a[5]                   # Fail - index out of range
```

`a' is a function
with domain {0,1,2}

- **Idea: an array is a function with a finite domain**
- Array indexing **is** a function call; it is not overloaded notation
- (This is why function call is $f[x]$ is written with square brackets.)
- If you call a (closed) function outside its domain, it fails

Array bounds checks

```
a[5] # Fail - index out of range
```

- How can we **test** for out of bounds?

```
if (a[5]) {"Yes"} else {"No"} # Test out of bounds
```

- How can we **use** the value?

```
a := (5,2,99);  
if (x:=a[1]) {x+1} else {0} # Returns 3
```

x is bound in the
predicate

x is in scope here...

...but not here

Closed functions with ~~finite~~ domains

- If you call **any** function, even one with an infinite domain, outside its domain, it fails

```
a:=(4,2,7)      # An array of 3 elements
a[5]            # Fail - index out of range

f(x:int):=x+1    # A function whose domain is int
f[ "wombat" ]    # Fails - argument outside domain
```



Not a type error

Finite functions can do narrowing

```
a := (3, 7, 4) ;  
i : int ; a[i] + 1
```

You can think of `i:int` as bringing a new existential variable into scope

- What values can `i` take? Clearly just 0,1,2!
- So expand `a[i]` to those three choices
- This is narrowing in Verse

```
a := (3, 7, 4) ;  
i : int ; a[i] + 1
```

=

```
a := (3, 7, 4) ;  
i : int ; ((i = 0 ; a[i] + 1) |  
          (i = 1 ; a[i] + 1) |  
          (i = 2 ; a[i] + 1))
```

=

```
4 | 8 | 5
```

Narrowing in action

```
a:=(3,7,4);  
i:int; i>0; a[i]+1
```

```
= 8 | 5
```



This cuts out the choice $i=0$

Narrowing in action

```
a := (3, 7, 4)
```

```
i: int; a[i]
```

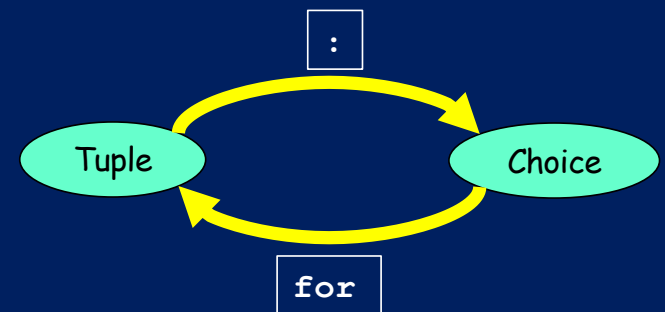
```
= i: int; (i=0; a[i]) |  
          (i=1; a[1]) |  
          (i=2; a[i])
```

```
= 3 | 7 | 4
```

: enumerates the
range of its argument

```
: a
```

```
= 3 | 7 | 4
```



for as forall

for{ $x:a$; $x+7$ } # Add 7 to each element of a

- Reads as $\forall x \in a. \text{blah}$
- But how does it work?

$x:t$ is syntactic sugar for $x := (:t)$

for{ $x := :a$; $x+7$ }

Choice of all the
elements of t

It's not just arrays that have finite domain!

```
f1 (1) <closed>      := 2      # Maps 1 ↦ 2, fails elsewhere  
f2 (1|2) <closed>    := 77     # Maps 1 ↦ 77, 2 ↦ 77  
f3 (x := 1|2) <closed> := x+7   # Maps 1 ↦ 8, 2 ↦ 9
```

All functions describe their domains in Verse

- Outside the domain, the function fails.
Thus `f2 [7]` fails.
- Narrowing works for functions too.

```
for{ f2 [ x:int ] ; x } # Returns (1,2)
```

All functions describe their domains

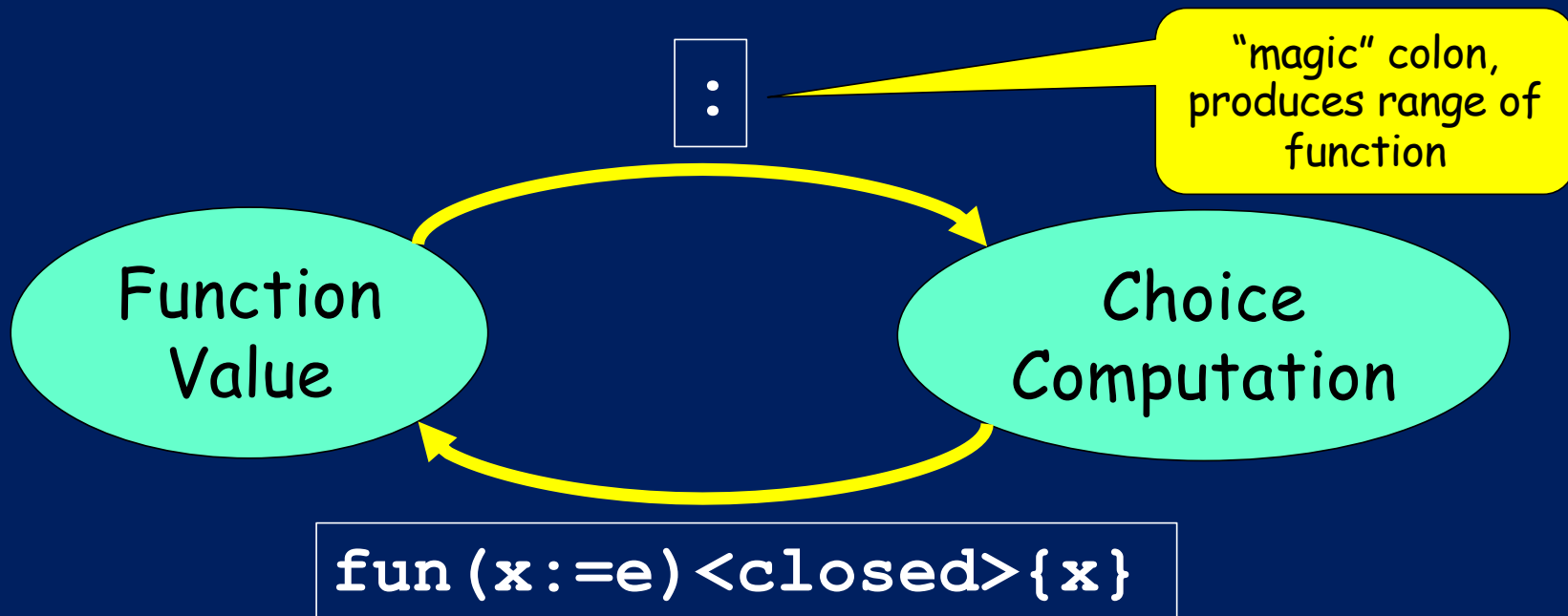
```
f1 (1) <closed>      := 2      # 1
f2 (1|2) <closed>    := 77     # 1|2
f3 (x := 1|2) <closed> := x+7   # 1|2
f4 (x:nat) <closed>   := x+1    # all nats
f5 (x:any) <closed>   := x      # "all" values
```

Domain may be infinite

Predicative semantics: domain
include f1-f4 but not f5 (or
other functions that use any)

From choice to functions & back

```
f (x := 4 | 2 | 7) <closed> := x    # Maps 4 ↦ 4, 2 ↦ 2, 7 ↦ 7  
:f                                # Behaves like 4 | 2 | 7
```



From unordered choice to functions & back

```
f (x := 4 || 2 || 7) <closed> := x    # Maps 4 ↦ 4, 2 ↦ 2, 7 ↦ 7  
: f                                # Behaves like 4 || 2 || 7
```

unordered choice

:

"magic" colon,
produces range of
function

Function
Value

Any Choice
Computation

```
fun (x := e) <closed> { x }
```

Types are expressions

Types in Verse

- In Verse, a **“type”** is simply a (partial) function
 - Function fails on values outside (the domain of) that type
 - E.g. The type `int` is a **function** that fails on strings and floats and is the identity function on integers.
 - e.g. `int[4] = 4`, but `int[“wombat”]` fails

- A function applies its parameter **type** to its argument **value**:

```
(x:int) => e
```

desugars to

```
λi. x:=int[i]; e
```

- A “type” is not necessarily an identity function

```
(x:abs) => e
```

desugars to

```
λi. x:=abs[i]; e
```

Types in Verse

- It is easy to define your own “types”

```
bit(x:=0|1)<closed> := x
```

The type that
contains 0 or 1

A call to this function fails
if the argument *x* is not a
natural number

```
nat(x:int)<closed><decides> := (x>=0)
```

- And use them in other definitions

```
not(x:bit)<closed> := if (x=0){1}else{0}
```

Types in Verse

- Types can easily be “dependent”

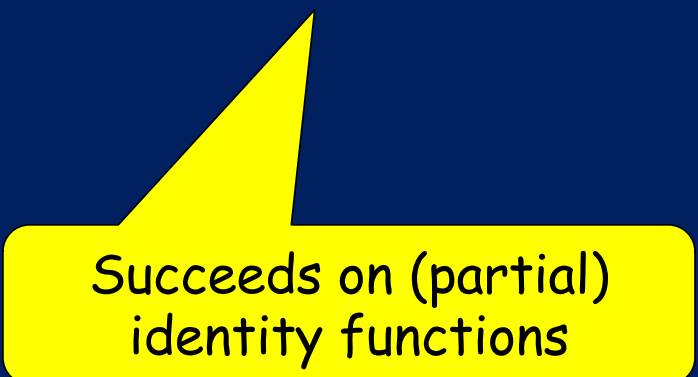
```
grPair (p:int, q:int) <closed> <decides>  
  := p > q; (p, q)
```

Type of q depends on value of p ,
namely q is drawn from all values less than p

Types in Verse

- Functions can be polymorphic

```
swap (t: type, x: t, y: t) <closed> := (y, x)
```



Succeeds on (partial)
identity functions

```
swap [int, 3, 7]
```

Towards a Denotational Semantics of Verse

(Work in progress!)

Goal: semantics for simple, total, Verse-like language with *unordered* choice “Timbda 0”

Semantics for untyped lambda calculus

$$V = \mathbb{N} + (V \rightarrow V)$$

$$\llbracket _ \rrbracket : \text{Env} \rightarrow V$$

$$\llbracket x \rrbracket \rho = \rho(x)$$

$$\llbracket \lambda x. e \rrbracket \rho = \lambda a:V. \llbracket e \rrbracket (\rho, x \mapsto a)$$

$$\llbracket e1 \ e2 \rrbracket \rho = (\llbracket e1 \rrbracket \rho) (\llbracket e2 \rrbracket \rho)$$

$$\llbracket k \rrbracket \rho = k$$

- LHS: syntax trees
RHS: ????

- **Problem: what is V ?** There is no set satisfying

$$V = V \rightarrow V$$

It is too big!

- Answer (Scott et al): **domain theory**

Domain Semantics for FLP

$$V = \mathbb{N} + (V \rightarrow V)$$

$$\llbracket _ \rrbracket : \text{Env} \rightarrow V$$

$$\llbracket x \rrbracket \rho = \rho(x)$$

$$\llbracket \lambda x. e \rrbracket \rho = \lambda a:V. \llbracket e \rrbracket (\rho, x \mapsto a)$$

$$\llbracket e1 \ e2 \rrbracket \rho = (\llbracket e1 \rrbracket \rho) (\llbracket e2 \rrbracket \rho)$$

$$\llbracket k \rrbracket \rho = k$$

- Applicable to FLP: see Jones et al. POPL 2026
- Domain theory requires a lot of structure: monotonicity, continuity, limits, and fixpoints
- Non-termination and bottom deeply built in; but we want to explore total functions and proofs.
(This is a choice.)

Typed semantics for typed lambda calculus

$$\llbracket \text{Nat} \rrbracket = \mathbb{N}$$

$$\llbracket \tau_1 \rightarrow \tau_2 \rrbracket = \llbracket \tau_1 \rrbracket \rightarrow \llbracket \tau_2 \rrbracket$$

$$\llbracket _ \rrbracket : \llbracket \Gamma \rrbracket \rightarrow \llbracket \tau \rrbracket$$

$$\llbracket x \rrbracket \rho = \rho(x)$$

$$\llbracket \lambda x:\tau. e \rrbracket \rho = \lambda a:\llbracket \tau_1 \rrbracket. \llbracket e \rrbracket (\rho, x \mapsto a)$$

$$\llbracket e_1 e_2 \rrbracket \rho = (\llbracket e_1 \rrbracket \rho) (\llbracket e_2 \rrbracket \rho)$$

$$\llbracket k \rrbracket \rho = k$$

- If we only care about total computations, why not embed well-typed terms into Agda/Rocq/Lean
- No function quantifies over itself
- Can extend to effectful languages with monads
- ... but only gives a semantics to typed terms, and Verse is untyped...

Set theoretic semantics for STLC

$$T[\text{Nat}] = \omega$$

$$T[\tau_1 \rightarrow \tau_2] = \Pi T[\tau_1] T[\tau_2]$$

where $\Pi A B$ = set of total functions from A to B

$$\llbracket _ \rrbracket : \text{Env} \rightarrow \text{Set}$$

$$\llbracket x \rrbracket \rho = \rho(x)$$

$$\llbracket \lambda x:\tau. e \rrbracket \rho = \{ \langle a, \llbracket e \rrbracket (\rho, x \mapsto a) \rangle \mid a \in T[\tau] \}$$

$$\llbracket e_1 e_2 \rrbracket \rho = \llbracket e_1 \rrbracket \rho \cdot \llbracket e_2 \rrbracket \rho$$

$$\llbracket k \rrbracket \rho = \underline{k}$$

Encoding of a function as a set of pairs (i.e. a graph)

Set-theoretic function application

Encoding of a natural number as a set

Set theoretic semantics for STLC

$$T[\text{Nat}] = \omega$$

$$T[\tau_1 \rightarrow \tau_2] = \Pi T[\tau_1] T[\tau_2]$$

where $\Pi A B$ = set of total functions from A to B

$$\llbracket _ \rrbracket : \text{Env} \rightarrow \text{Set}$$

$$\llbracket x \rrbracket \rho = \rho(x)$$

$$\llbracket \lambda x:\tau. e \rrbracket \rho = \{ \langle a, \llbracket e \rrbracket (\rho, x \mapsto a) \rangle \mid a \in T[\tau] \}$$

$$\llbracket e_1 e_2 \rrbracket \rho = \llbracket e_1 \rrbracket \rho \cdot \llbracket e_2 \rrbracket \rho$$

$$\llbracket k \rrbracket \rho = \underline{k}$$

Need to know domain of the function, i.e. set of values with this type

Set theoretic semantics for STLC

$$\llbracket x \rrbracket \rho = \rho(x)$$

$$\llbracket \lambda x:\tau. e2 \rrbracket \rho = \{ \langle a, \llbracket e \rrbracket(\rho, x \mapsto a) \rangle \mid a \in T[\tau] \}$$

$$\llbracket e1 \ e2 \rrbracket \rho = \llbracket e1 \rrbracket \rho \cdot \llbracket e2 \rrbracket \rho$$

$$\llbracket k \rrbracket \rho = \underline{k}$$

In Verse, this is an expression
not a type

- Key point: the type on a lambda binder limits the values that the binder can take on
- **Key benefit: All expressions have denotations, including ill-typed ones**

Semantics as sets of values

$$\llbracket x \rrbracket \rho = \{ \rho(x) \}$$

$$\llbracket \lambda(x:\tau). e2 \rrbracket \rho = \Pi T[\tau] (\lambda a. \llbracket e2 \rrbracket (\rho, x \mapsto a))$$

$$\llbracket e1 \ e2 \rrbracket \rho = f \leftarrow \llbracket e1 \rrbracket \rho; v \leftarrow \llbracket e2 \rrbracket \rho; f \cdot v$$

$$\llbracket k \rrbracket \rho = \{ \underline{k} \}$$

...

where

$\Pi A B$ = set of total functions from A to B (dep)

$x \leftarrow A ; B$ = union of sets in B for every x in A

Semantics as sets of values

$$\begin{aligned}\llbracket x \rrbracket \rho &= \{ \rho(x) \} \\ \llbracket \lambda x:\tau. e2 \rrbracket \rho &= \Pi T[\tau] \ (\ \lambda a. \llbracket e2 \rrbracket (\rho, x \mapsto a) \) \\ \llbracket e1 \ e2 \rrbracket \rho &= f \leftarrow \llbracket e1 \rrbracket \rho; v \leftarrow \llbracket e2 \rrbracket \rho; f \cdot v \\ \llbracket k \rrbracket \rho &= \{ \underline{k} \}\end{aligned}$$

- Denotation of a term is a **set** of values; always **empty** or **singleton**.
- Every term (ill-typed or not) has a denotation
- Empty-set means “ill typed term”; e.g. $(3 \ 4)$.
- But not vice versa: some “ill typed terms” have non-empty denotations

"x:e" is "x:= :e"

Verse semantics as sets of values

$$\llbracket x \rrbracket \rho = \{ \rho(x) \}$$

$$\llbracket \lambda x := e1. e2 \rrbracket \rho = \Pi \llbracket e1 \rrbracket (\lambda a. \llbracket e2 \rrbracket (\rho, x \mapsto a))$$

$$\llbracket e1 \ e2 \rrbracket \rho = f \leftarrow \llbracket e1 \rrbracket \rho; v \leftarrow \llbracket e2 \rrbracket \rho; f \cdot v$$

$$\llbracket k \rrbracket \rho = \{ \underline{k} \}$$

$$\llbracket :e \rrbracket \rho = f \leftarrow \llbracket e \rrbracket \rho; \{ y \mid \exists x. \langle x, y \rangle \in f \}$$

$$\llbracket \text{nat} \rrbracket \rho = \{ \langle x, x \rangle \mid x \in \omega \}$$

an expression

range of f

Identity function on nat

- Denotation of a term could be an infinite set, i.e. $\llbracket \text{nat} \rrbracket$ is ω
- $\llbracket \lambda x:\text{nat}. : \text{nat} \rrbracket = \Pi \llbracket : \text{nat} \rrbracket \llbracket : \text{nat} \rrbracket$ is set of all total functions from ω to ω

Verse semantics as sets of values

...

$$\llbracket e1 \ e2 \rrbracket \rho = f \leftarrow \llbracket e1 \rrbracket \rho; v \leftarrow \llbracket e2 \rrbracket \rho; f \cdot v$$

$$\llbracket k \rrbracket \rho = \{ \underline{k} \}$$

$$\llbracket :e \rrbracket \rho = f \leftarrow \llbracket e \rrbracket \rho; \{ y \mid \exists x. \langle x, y \rangle \in f \}$$

$$\llbracket \text{Nat} \rrbracket \rho = \{ \langle x, x \rangle \mid x \in \omega \}$$

$$\llbracket \text{fail} \rrbracket \rho = \emptyset$$

$$\llbracket e1 \parallel e2 \rrbracket \rho = \llbracket e1 \rrbracket \rho \cup \llbracket e2 \rrbracket \rho$$

$$\llbracket e1 = e2 \rrbracket \rho = \llbracket e1 \rrbracket \rho \cap \llbracket e2 \rrbracket \rho$$

$$\llbracket \exists x := e1. e2 \rrbracket \rho = x \leftarrow \llbracket e1 \rrbracket \rho; \llbracket e2 \rrbracket (\rho, x \mapsto a)$$

Unordered choice

Unification

Existential

Towards Verse: “Timbda 0”

- Remarkably simple – even simpler than STLC, unified functions and arrow types
- Just ZF set theory. No domains.
- No type system: every term has a denotation.
- “Ill typed” terms simply denote the empty set, e.g. $(3\ 4)$
- Full Verse requires similar ideas, but much more structure (i.e. ordered choice, pattern matching, etc.)

Wrapping up